人工神经网络

本章目录

01 发展历史

02 感知机算法

03 BP算法

1.人工神经网络发展历史

01 发展历史

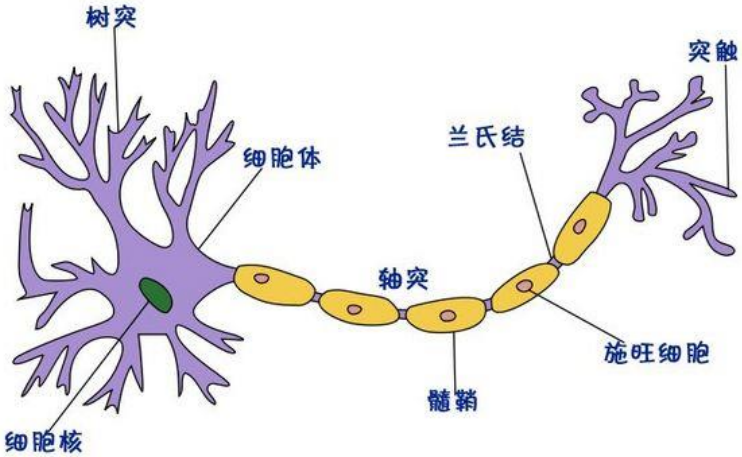
02 感知机算法

03 BP算法

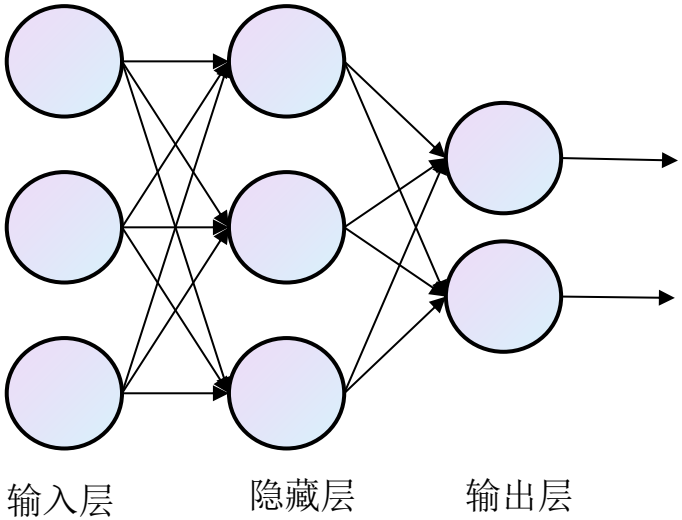
1.人工神经网络发展历史

发展历史

1943年，心理学家McCulloch和逻辑学家Pitts建立神经网络的数学模型，MP模型



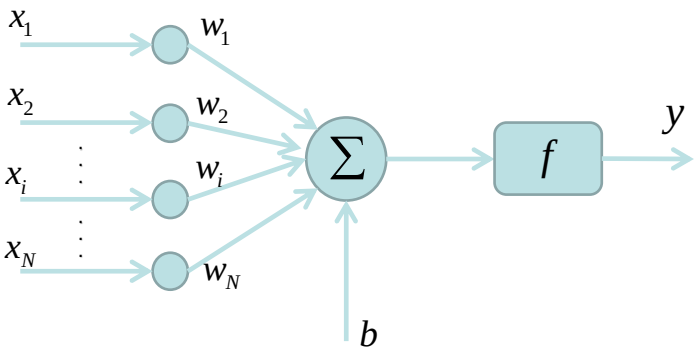
神经元生理结构



神经元数学模型

1.人工神经网络发展历史

1960年代，人工网络得到了进一步地发展感知机和自适应线性元件等被提出。M.Minsky仔细分析了以感知机为代表的神经网络的局限性，指出了感知机不能解决非线性问题，这极大影响了神经网络的研究。



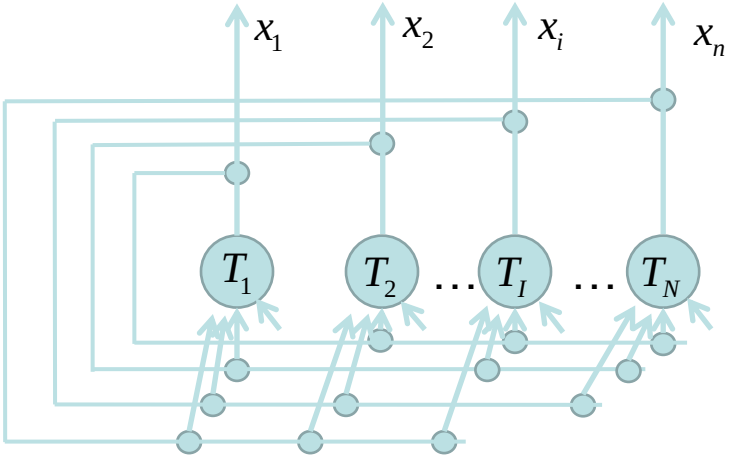
$$y = f\left(\sum_{i=1}^{N} w_i x_i + b\right)$$

单层感知机的数学模型

1.人工神经网络发展历史

1982年，加州理工学院

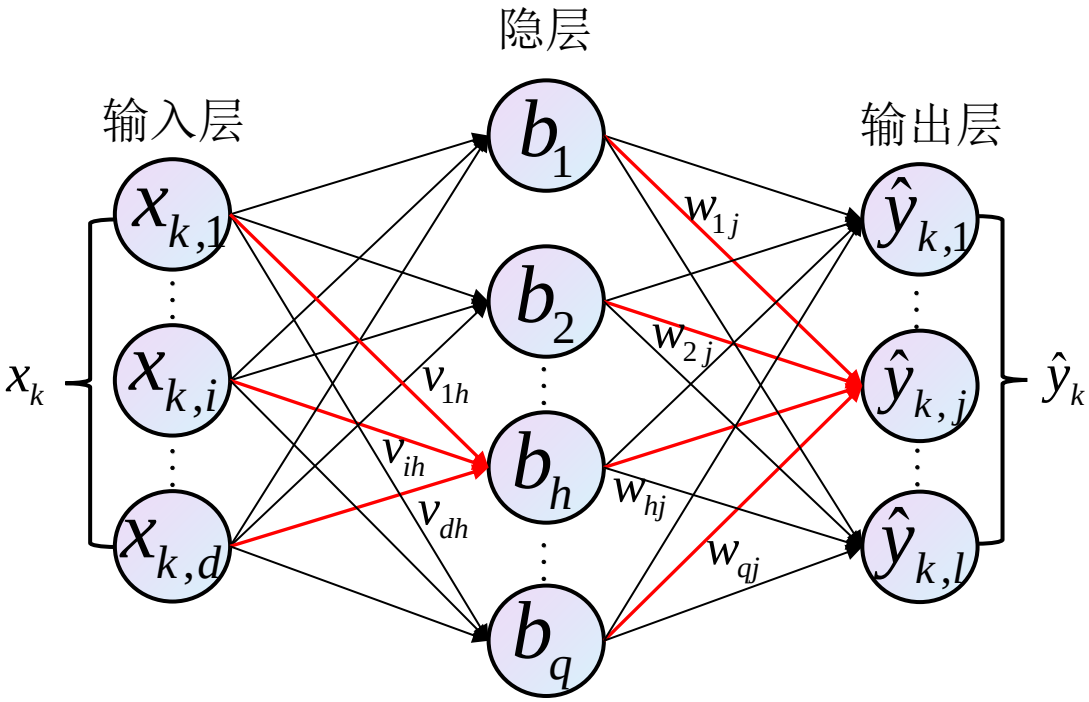
J.J.Hopfield教授提出了Hopfield神经网络模型，引入了计算能量概念，给出了网络稳定性判断。



离散Hopfield神经网络模型

1.人工神经网络发展历史

1986年，Rumelhart和McClelland为首的科学家提出了BP（Back Propagation）神经网络的概念，是一种按照误差逆向传播算法训练的多层前馈神经网络，目前是应用最广泛的神经网络。



BP神经网络模型

1.人工神经网络发展历史

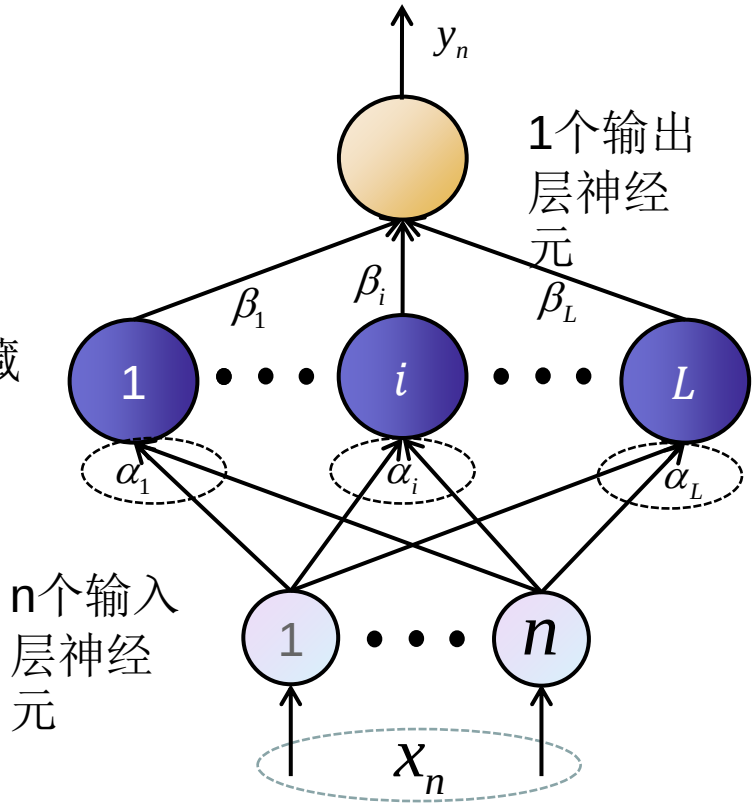
极限学习机(Extreme Learning Machine, ELM)，是由黄广斌提出的用于处理单隐层神经网络的算法

随机初始化输入权重 α_i 和偏置，只求解输出权重值β_i。

优点：

1.学习精度有保证

2.学习速度快



2.感知器算法

01 发展历史

02 感知机算法

03 BP算法

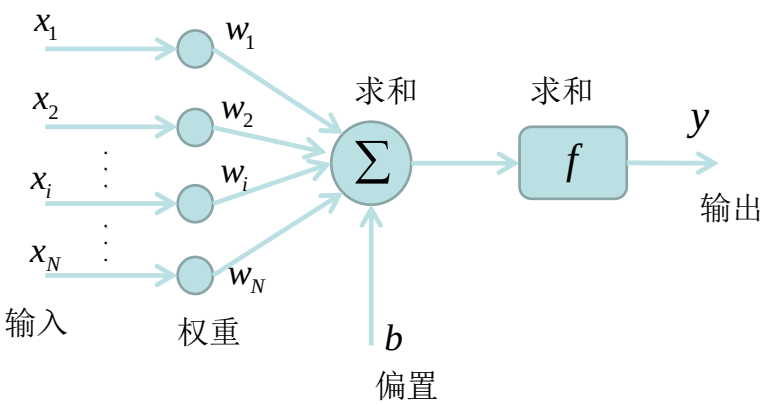
2.感知机算法

感知机（Perceptron）是二分类问题的线性分类模型。

用 $X \in R^{n\times d}$ 表示数据集，用 Y 表示标签。

需要学习的目标函数是

$$f(x) = \mathrm{sign}(w^T x + b)$$



从一堆输入输出中学习模型参数w和b。

2.感知机算法

感知机算法（Perceptron Algorithm）:

随机选择模型参数的(w_0, b_0)初始值。

选择一个训练样本(x_n, y_n)。

若判别函数$w^T x_n + b > 0$，且$y_n = -1$，则$w = w - x_n$，$b = b - 1$。

若判别函数$w^T x_n + b < 0$，且$y_n = +1$，则$w = w + x_n$，$b = b + 1$。

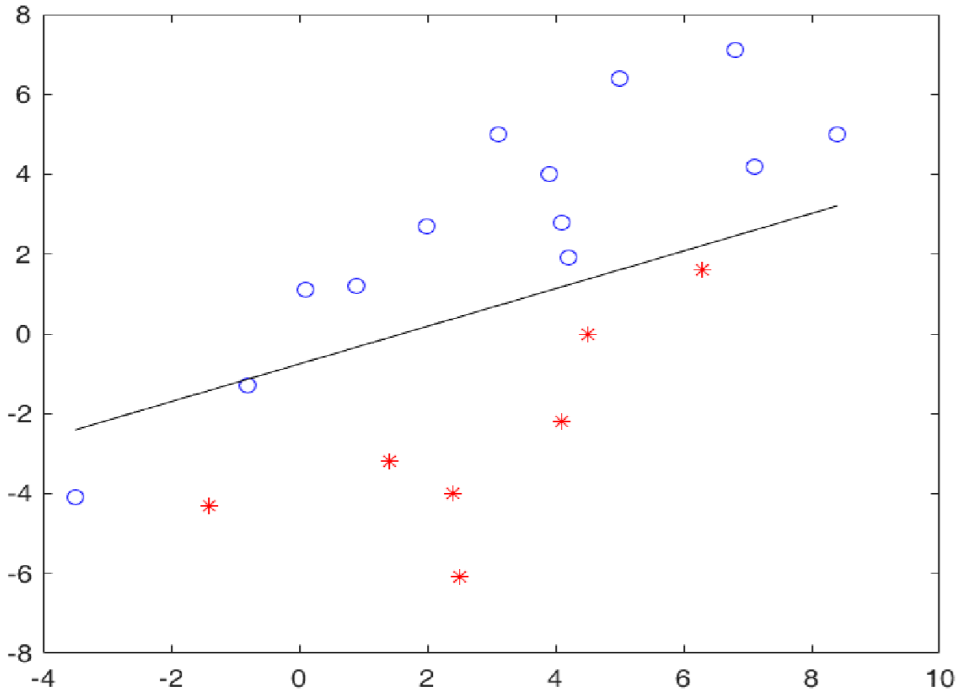
再选取另一个训练样本(x_m, y_m)，回到2。

终止条件：直到所有数据的输入输出对都不满足2中的(i)和(ii)中之一，则退出循环。

2.感知机算法

算法演示　分类问题

单层感知机只能处理线性问题，无法处理非线性问题！！



2.感知器算法

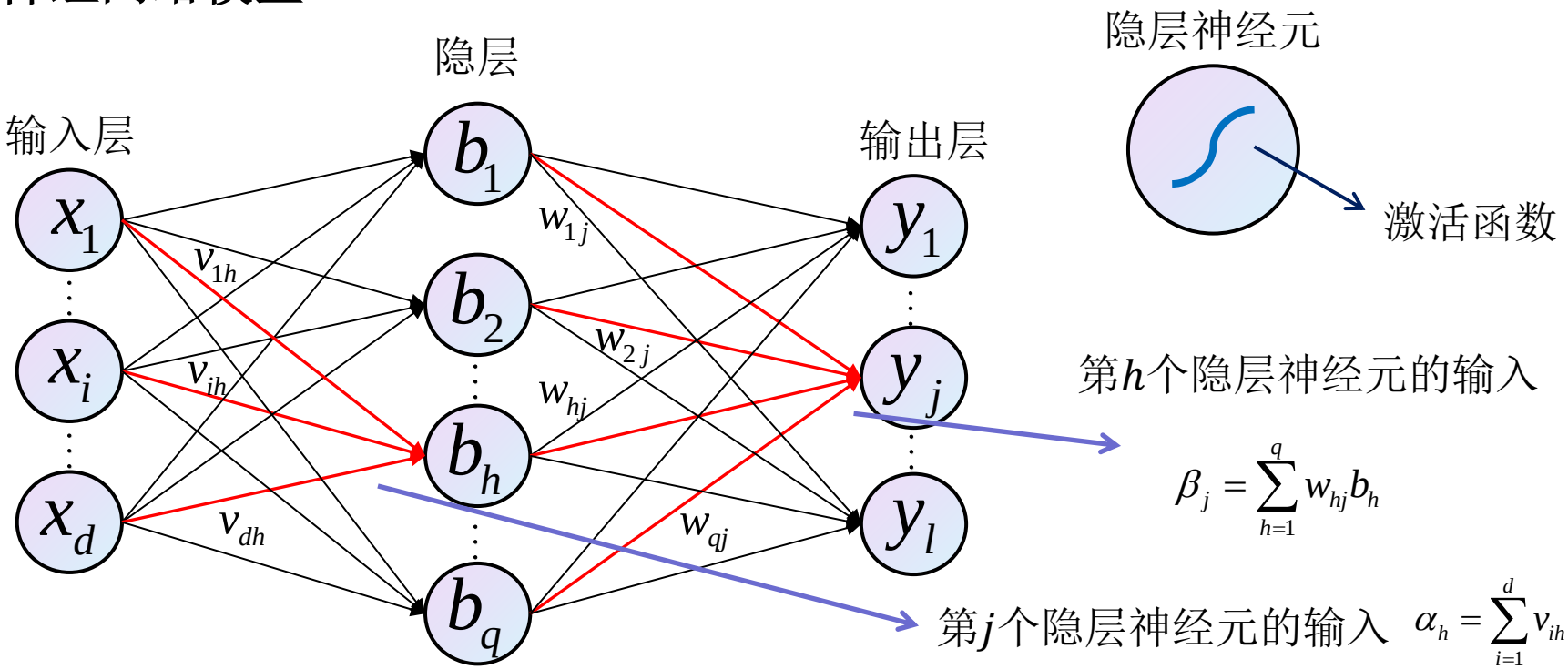
01 发展历史

02 感知机算法

03 BP算法

3.BP算法

神经网络模型

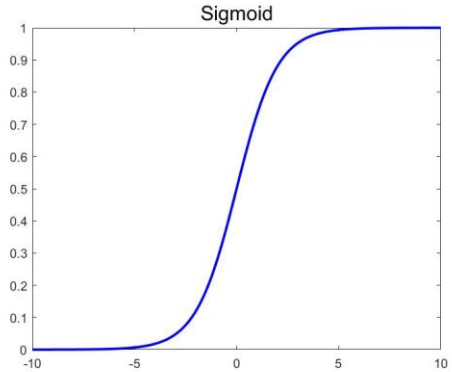


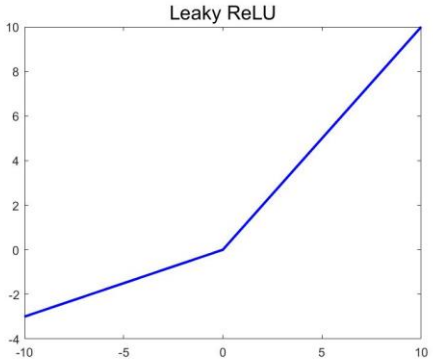
3.BP算法

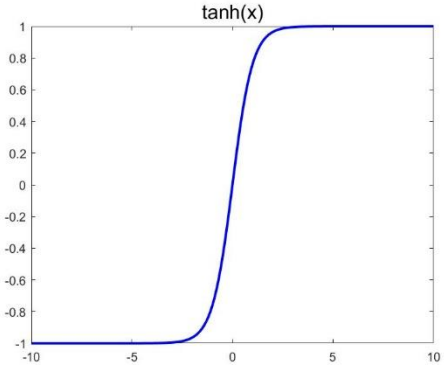
激活函数

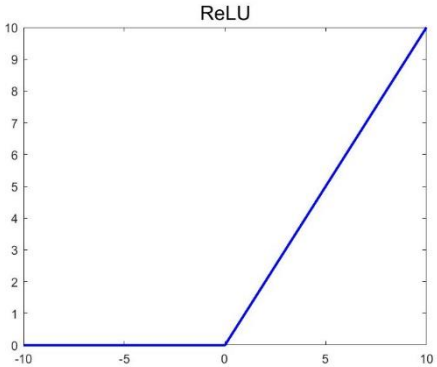
常见激活函数选择：

sigmoid 函数
tanh 函数
ReLU 函数
Leaky ReLU函数









3.BP算法

最常用Sigmoid函数的优缺点：

优点：
1.函数处处连续，便于求导
2.可将函数值的范围压缩至[0,1]，可用于压缩数据，且幅度不变
3.便于前向传输

缺点：
1.在趋向无穷的地方，函数值变化很小，容易出现梯度消失，不利于深层神经的反馈传输
2.幂函数的梯度计算复杂
3.收敛速度比较慢

3.BP算法

主要步骤

第一步，对样本明确预测输出值与损失函数

第二步，明确参数调整策略

第三步，计算输出层阈值的梯度

第四步，计算隐层到输出层连接权值的梯度

第五步，计算隐层阈值的梯度

第六步，计算输入层到隐层连接权值的梯度

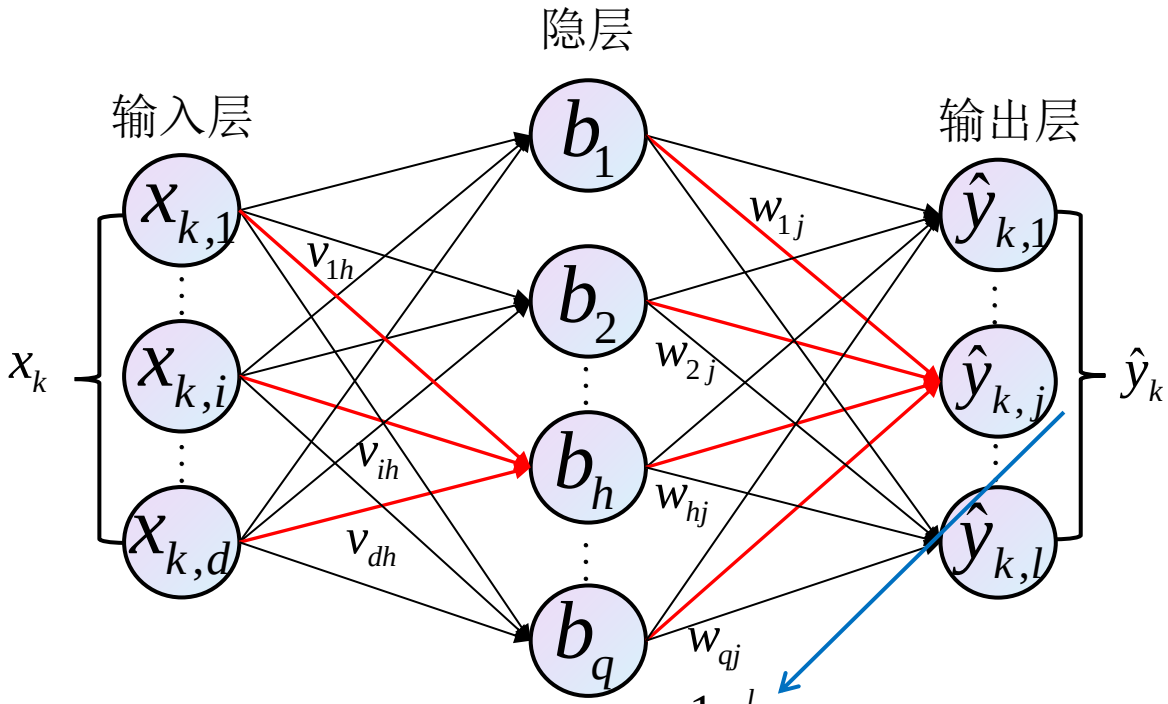
第七步，引出归纳结论

3.BP算法

第一步，明确损失函数

对样本(x_k, y_k)，神经网络的预测输出值为$\hat{y}_k$。

全网络在样本(x_k, y_k)上的均方误差 $E_k = \frac{1}{2}\sum_{j=1}^{l}\left(\hat{y}_j^k - y_j^k\right)^2$



预测均方误差 $E_k = \frac{1}{2}\sum_{j=1}^{l}\left(\hat{y}_j^k - y_j^k\right)^2$

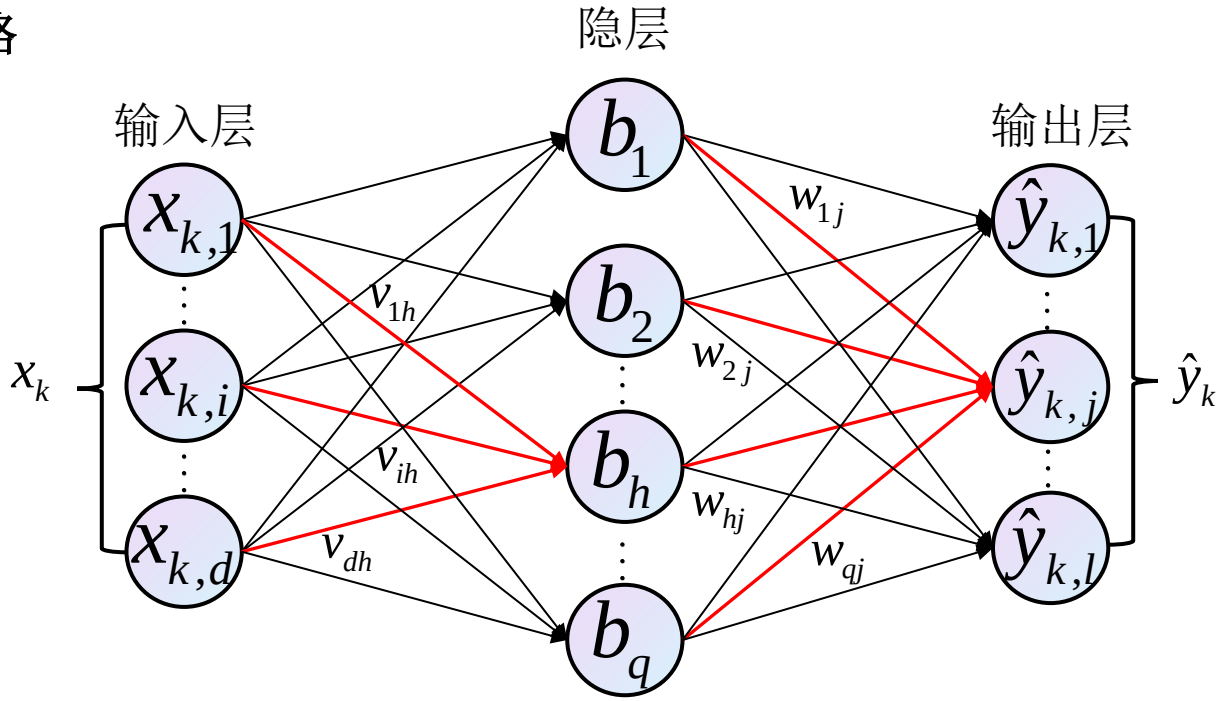
3.BP算法

第二步，明确参数调整策略

基于梯度下降（Gradient Descent）策略，以目标的负梯度方向对参数进行调整

$$v = v + \Delta v$$

$$\Delta v = -\rho \frac{\partial E_n}{\partial v}$$



3.BP算法

第三步，计算输出层阈值$\boldsymbol{\theta_j}$的梯度$\frac{\partial E_k}{\partial \theta_j}$

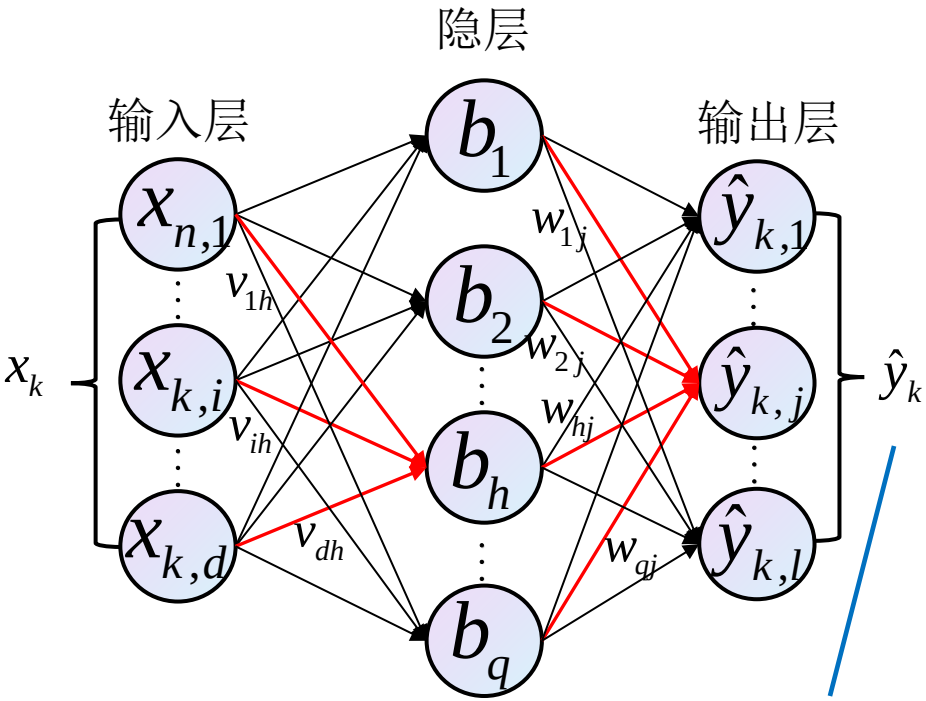
利用链式法则，可得

$$\frac{\partial E_k}{\partial \theta_j} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \theta_j}$$

其中，$\frac{\partial E_k}{\partial \hat{y}_j^k} = \hat{y}_j^k - y_j^k$ $\quad \frac{\partial \hat{y}_j^k}{\partial \theta_j} = -\hat{y}_j^k\left(1-\hat{y}_j^k\right)$

所以，$g_j = \frac{\partial E_k}{\partial \theta_j} = \hat{y}_j^k\left(1-\hat{y}_j^k\right)\left(y_j^k - \hat{y}_j^k\right)$

更新公式 $\theta_j \leftarrow \theta_j - \eta g_j$



对阈值求导 $\frac{\partial E_k}{\partial \theta_j}$

3.BP算法

第四步，计算隐层到输出层连接权值w_{hj}的梯度$\frac{\partial E_k}{\partial w_{hj}}$

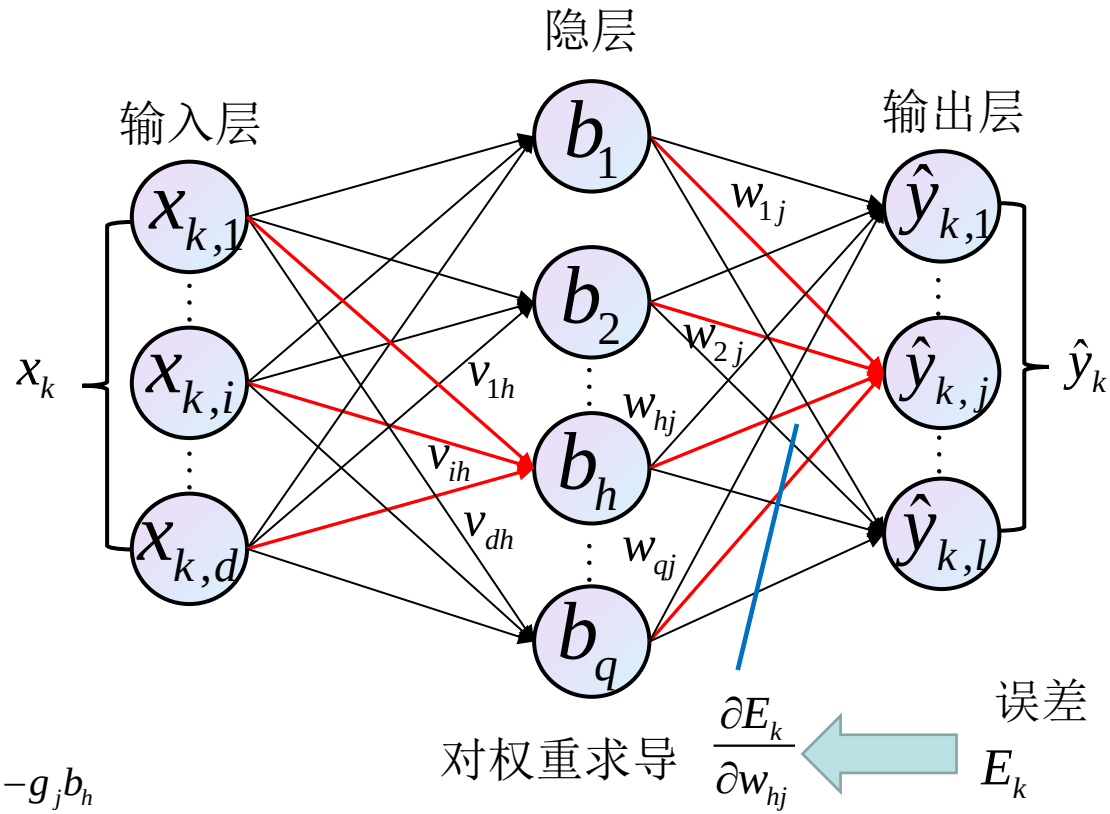
利用链式法则，可得

$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}}$$

其中，$\frac{\partial E_k}{\partial \hat{y}_j^k} = \hat{y}_j^k - y_j^k$ $\quad \frac{\partial \hat{y}_j^k}{\partial \beta_j} = \hat{y}_j^k\left(1-\hat{y}_j^k\right)$

可得 $\frac{\partial \beta_j}{\partial w_{hj}} = b_h$

综上可得$\frac{\partial E_k}{\partial w_{hj}} = \hat{y}_j^k \cdot \left(\hat{y}_j^k - y_j^k\right) \cdot \left(1-\hat{y}_j^k\right) \cdot b_h = -g_j b_h$



3.BP算法

第五步，计算隐层阈值γ_h的梯度$\frac{\partial E_k}{\partial \gamma_h}$

利用链式法则，可得

$$\frac{\partial E_k}{\partial \gamma_h}=\frac{\partial E_k}{\partial b_h}\cdot\frac{\partial b_h}{\partial \gamma_h}$$

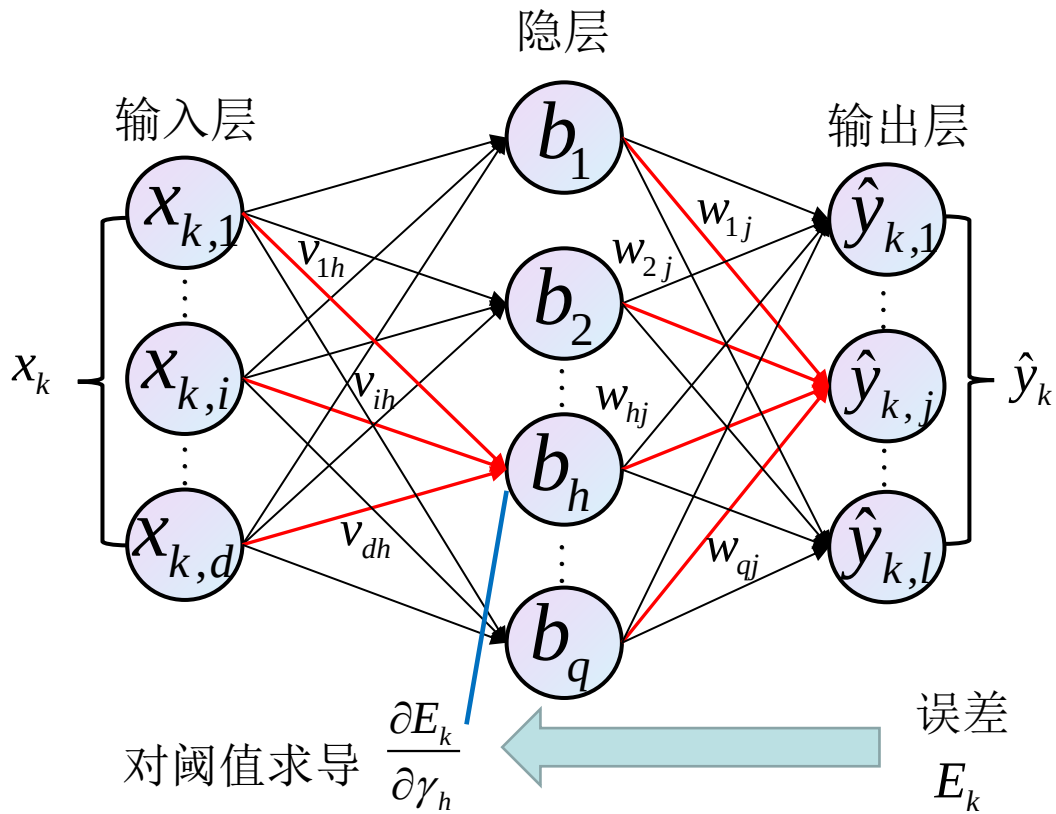
其中，$\frac{\partial E_k}{\partial b_h}=\sum_{j=1}^{l}\frac{\partial E_k}{\partial \hat{y}_j^k}\cdot\frac{\partial \hat{y}_j^k}{\partial \beta_j}\cdot\frac{\partial \beta_j}{\partial b_h}=-\sum_{j=1}^{l}g_j w_{hj}$

$$\frac{\partial b_h}{\partial \gamma_h}=\frac{\partial}{\partial \gamma_h}f(\alpha_h-\gamma_h)=-b_h(1-b_h)$$

所以有 $\frac{\partial E_k}{\partial \gamma_h}=b_h(1-b_h)\sum_{j=1}^{l}w_{hj}g_j$

令 $e_h=b_h(1-b_h)\sum_{j=1}^{l}w_{hj}g_j$

更新公式 $\gamma_h \leftarrow \gamma_h-\eta e_h$



3.BP算法

第六步，计算隐层权重v_{ih}的梯度$\frac{\partial E_k}{\partial v_{ih}}$

利用链式法则，可得

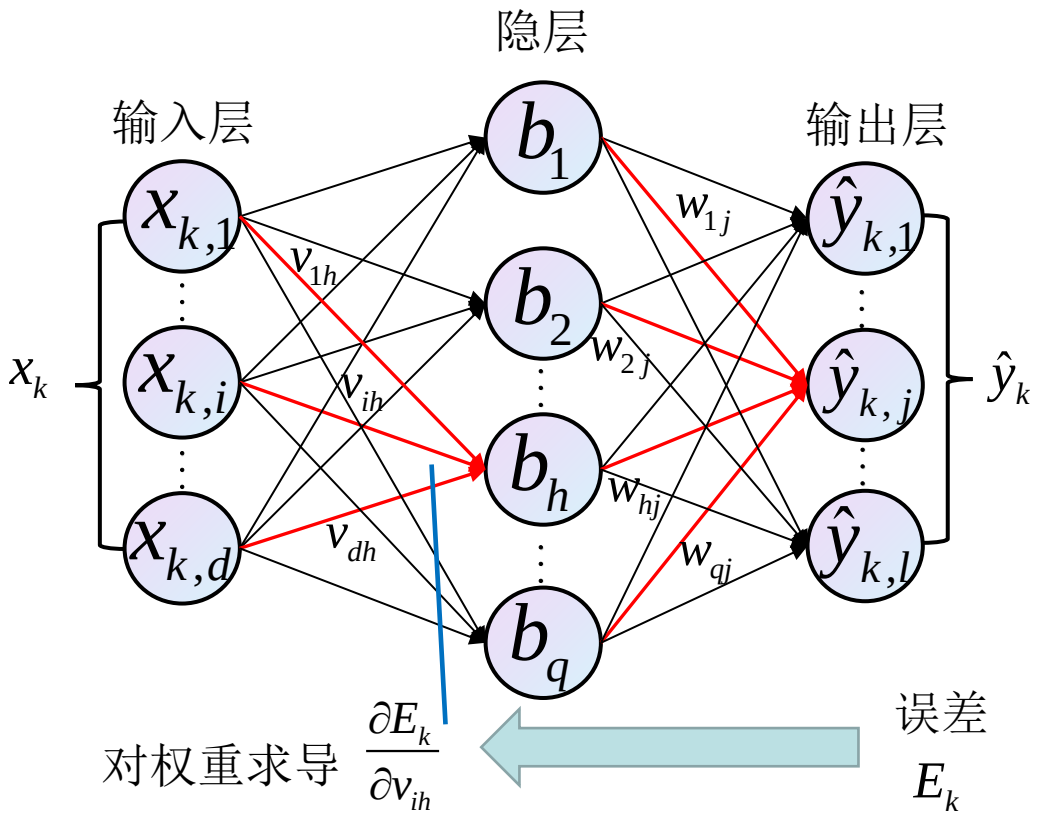
$$\frac{\partial E_k}{\partial v_{ih}} = \frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h} \cdot \frac{\partial \alpha_h}{\partial v_{ih}}$$

其中，$\frac{\partial E_k}{\partial b_h} = -\sum_{j=1}^{l} g_j w_{hj}$ $\quad \frac{\partial b_h}{\partial \alpha_h} = b_h(1-b_h)$

$$\frac{\partial \alpha_h}{\partial v_{ih}} = x_i$$

所以有$\frac{\partial E_k}{\partial v_{ih}} = -b_h(1-b_h)x_i \sum_{j=1}^{l} w_{hj} g_j = -e_h x_i$

更新公式 $v_{ih} \leftarrow v_{ih} + \eta e_h x_i$



3.BP算法

第七步，引出结论

观察$\frac{\partial E_k}{\partial \gamma_h} = b_h\,(1-b_h)\sum_{j=1}^{l} g_j w_{hj}$，可知

隐层阈值梯度取决于隐层神经元输出、输出层阈值梯度和隐层与输出层的连接权值。

在阈值的调整过程中，当前层的阈值梯度取决于下一层的阈值，这就是BP算法的精髓。

观察$\frac{\partial E_k}{\partial w_{hj}} = -g_j b_h$，可知

当前层的连接权值梯度，取决于当前神经元阈值梯度和上层神经元输出。

3.BP算法

第七步，引出结论

只要知道上一层神经元的阈值梯度，即可计算当前层神经元阈值梯度和连接权值梯度。

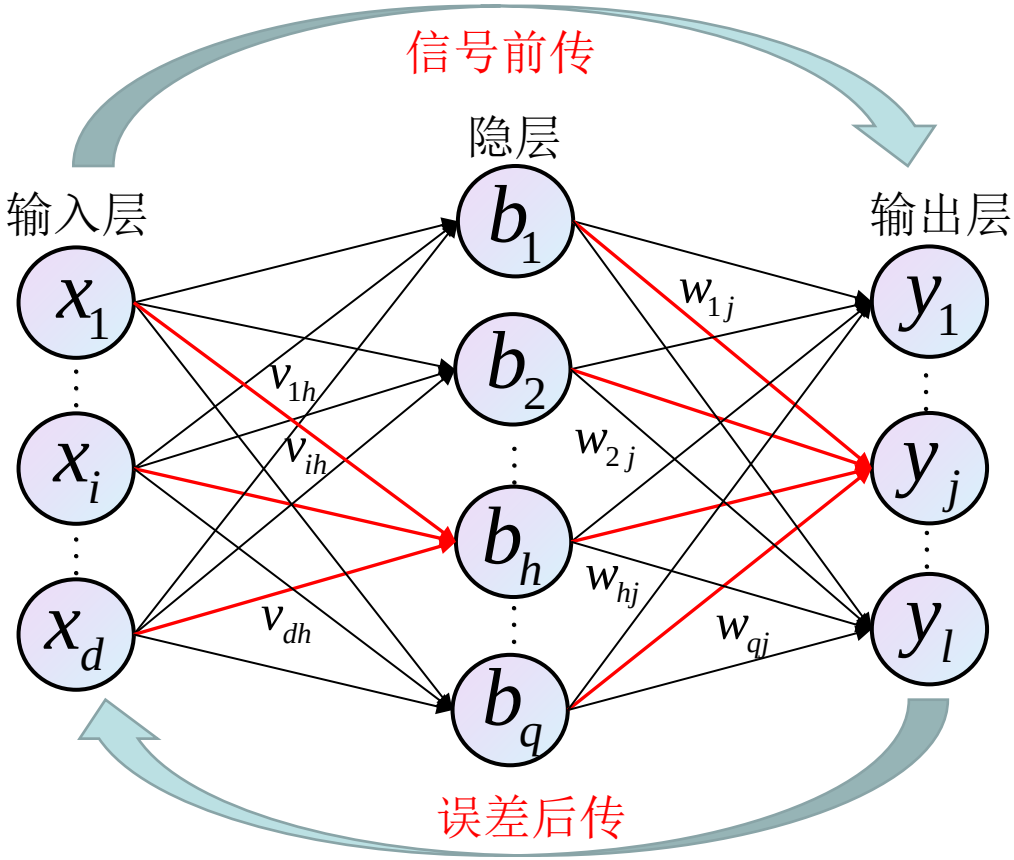
随后可以计算输出层神经元阈值梯度，从而计算出全网络的神经元阈值和连接权值梯度。

最终达到训练网络的目的。

3.BP算法

算法流程回顾：

1.将输入样本提供给输入层神经元
2.逐层将信号前传至隐层、输出层，产生输出层的结果
3.计算输出层误差
4.将误差反向传播至隐藏层神经元
5.根据隐层神经元对连接权重和阈值进行调整
6.上述过程循环进行，直至达到某些停止条件为止



3.BP算法

优点：

1.能够自适应、自主学习。BP可以根据预设参数更新规则，通过不断调整神经网络中的参数，已达到最符合期望的输出。

2.拥有很强的非线性映射能力。

3.误差的反向传播采用的是成熟的链式法则，推导过程严谨且科学。

4.算法泛化能力很强。

缺点：

1.BP神经网络参数众多，每次迭代需要更新较多数量的阈值和权值，故收敛速度比较慢。

2.网络中隐层含有的节点数目没有明确的准则，需要不断设置节点数字试凑，根据网络误差结果最终确定隐层节点个数

3.BP算法是一种速度较快的梯度下降算法，容易陷入局部极小值的问题。

利用多层感知机进行图像分类的Pytorch实现

3.8.1 隐藏层

多层感知机在单层神经网络的基础上引入了一到多个隐藏层（hidden layer）。隐藏层位于输入层和输出层之间。图3.3展示了一个多层感知机的神经网络图，它含有一个隐藏层，该层中有5个隐藏单元。

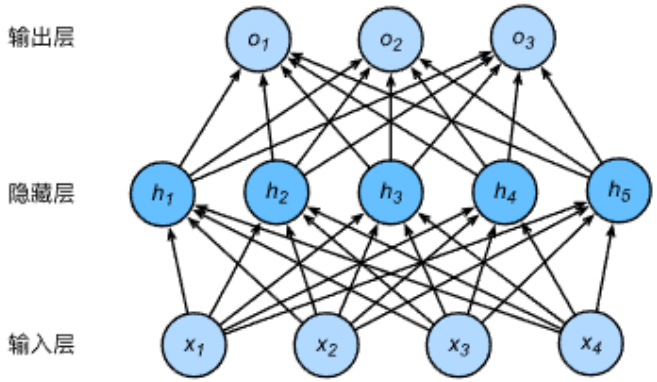


图3.3 带有隐藏层的多层感知机

在图3.3所示的多层感知机中，输入和输出个数分别为4和3，中间的隐藏层中包含了5个隐藏单元（hidden unit）。由于输入层不涉及计算，图3.3中的多层感知机的层数为2。由图3.3可见，隐藏层中的神经元和输入层中各个输入完全连接，输出层中的神经元和隐藏层中的各个神经元也完全连接。因此，多层感知机中的隐藏层和输出层都是全连接层。

具体来说，给定一个小批量样本$\boldsymbol{X} \in \mathbb{R}^{n\times d}$，其批量大小为$n$，输入个数为$d$。假设多层感知机只有一个隐藏层，其中隐藏单元个数为h。记隐藏层的输出（也称为隐藏层变量或隐藏变量）为$\boldsymbol{H}$，有$\boldsymbol{H} \in \mathbb{R}^{n\times h}$。因为隐藏层和输出层均是全连接层，可以设隐藏层的权重参数和偏差参数分别为$\boldsymbol{W}_h \in \mathbb{R}^{d\times h}$和$\boldsymbol{b}_h \in \mathbb{R}^{1\times h}$，输出层的权重和偏差参数分别为$\boldsymbol{W}_o \in \mathbb{R}^{h\times q}$和$\boldsymbol{b}_o \in \mathbb{R}^{1\times q}$。

我们先来看一种含单隐藏层的多层感知机的设计。其输出$\boldsymbol{O} \in \mathbb{R}^{n \times q}$的计算为

$$\boldsymbol{H} = \boldsymbol{X}\boldsymbol{W}_h + \boldsymbol{b}_h,$$
$$\boldsymbol{O} = \boldsymbol{H}\boldsymbol{W}_o + \boldsymbol{b}_o,$$

也就是将隐藏层的输出直接作为输出层的输入。如果将以上两个式子联立起来，可以得到

$$\boldsymbol{O} = (\boldsymbol{X}\boldsymbol{W}_h + \boldsymbol{b}_h)\boldsymbol{W}_o + \boldsymbol{b}_o = \boldsymbol{X}\boldsymbol{W}_h\boldsymbol{W}_o + \boldsymbol{b}_h\boldsymbol{W}_o + \boldsymbol{b}_o.$$

从联立后的式子可以看出，虽然神经网络引入了隐藏层，却依然等价于一个单层神经网络：其中输出层权重参数为$\boldsymbol{W}_h\boldsymbol{W}_o$，偏差参数为$\boldsymbol{b}_h\boldsymbol{W}_o + \boldsymbol{b}_o$。不难发现，即便再添加更多的隐藏层，以上设计依然只能与仅含输出层的单层神经网络等价。

3.8.2 激活函数

上述问题的根源在于全连接层只是对数据做仿射变换（affine transformation），而多个仿射变换的叠加仍然是一个仿射变换。解决问题的一个方法是引入非线性变换，例如对隐藏变量使用按元素运算的非线性函数进行变换，然后再作为下一个全连接层的输入。这个非线性函数被称为激活函数（activation function）。下面我们介绍几个常用的激活函数。

3.8.2.1 ReLU函数

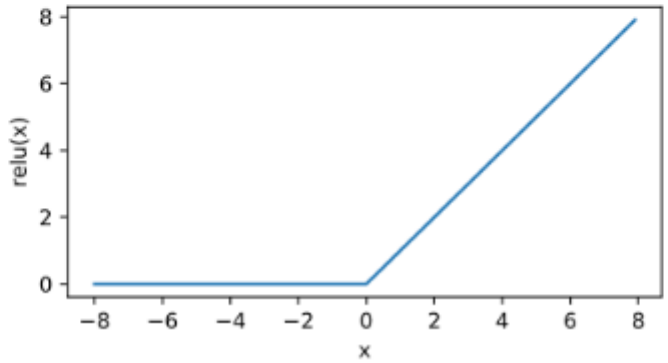
ReLU（rectified linear unit）函数提供了一个很简单的非线性变换。给定元素x，该函数定义为

$$\mathrm{ReLU}(x) = \max(x, 0).$$

可以看出，ReLU函数只保留正数元素，并将负数元素清零。为了直观地观察这一非线性变换，我们先定义一个绘图函数 `xyplot` 。

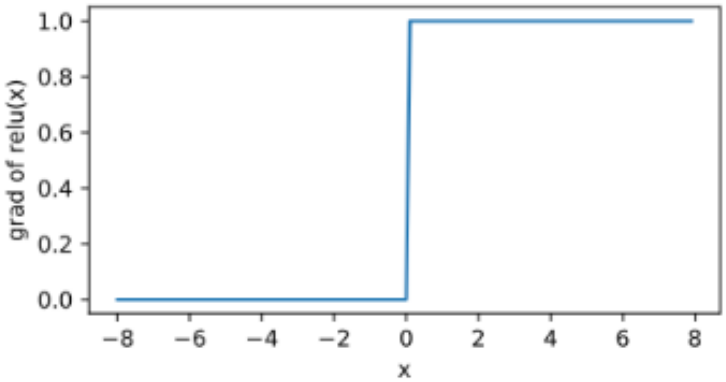
我们接下来通过 `Tensor` 提供的 `relu` 函数来绘制ReLU函数。可以看到，该激活函数是一个两段线性函数。

```python
x = torch.arange(-8.0, 8.0, 0.1, requires_grad=True)
y = x.relu()
xyplot(x, y, 'relu')
```



显然，当输入为负数时，ReLU函数的导数为0；当输入为正数时，ReLU函数的导数为1。尽管输入为0时ReLU函数不可导，但是我们可以取此处的导数为0。下面绘制ReLU函数的导数。

```python
y.sum().backward()
xyplot(x, x.grad, 'grad of relu')
```



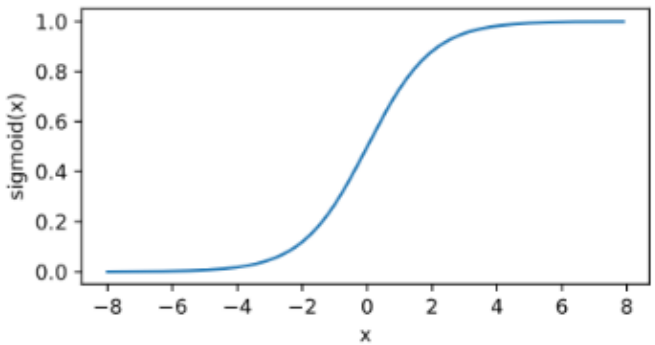
3.8.2.2 sigmoid函数

sigmoid函数可以将元素的值变换到0和1之间：

$$\mathrm{sigmoid}(x) = \frac{1}{1+\exp(-x)}.$$

sigmoid函数在早期的神经网络中较为普遍，但它目前逐渐被更简单的ReLU函数取代。在后面“循环神经网络”一章中我们会介绍如何利用它值域在0到1之间这一特性来控制信息在神经网络中的流动。下面绘制了sigmoid函数。当输入接近0时，sigmoid函数接近线性变换。

```python
y = x.sigmoid()
xyplot(x, y, 'sigmoid')
```



3.8.3 多层感知机

多层感知机就是含有至少一个隐藏层的由全连接层组成的神经网络，且每个隐藏层的输出通过激活函数进行变换。多层感知机的层数和各隐藏层中隐藏单元个数都是超参数。以单隐藏层为例并沿用本节之前定义的符号，多层感知机按以下方式计算输出：

$$\boldsymbol{H} = \phi(\boldsymbol{X}\boldsymbol{W}_h + \boldsymbol{b}_h),$$
$$\boldsymbol{O} = \boldsymbol{H}\boldsymbol{W}_o + \boldsymbol{b}_o,$$

其中ϕ表示激活函数。在分类问题中，我们可以对输出$\boldsymbol{O}$做softmax运算，并使用softmax回归中的交叉熵损失函数。 在回归问题中，我们将输出层的输出个数设为1，并将输出$\boldsymbol{O}$直接提供给线性回归中使用的平方损失函数。

3.4.5 交叉熵损失函数

前面提到，使用softmax运算后可以更方便地与离散标签计算误差。我们已经知道，softmax运算将输出变换成一个合法的类别预测分布。实际上，真实标签也可以用类别分布表达：对于样本i，我们构造向量$\boldsymbol{y}^{(i)} \in \mathbb{R}^q$，使其第$y^{(i)}$（样本$i$类别的离散数值）个元素为1，其余为0。这样我们的训练目标可以设为使预测概率分布$\hat{\boldsymbol{y}}^{(i)}$尽可能接近真实的标签概率分布$\boldsymbol{y}^{(i)}$。

我们可以像线性回归那样使用平方损失函数$\|\hat{\boldsymbol{y}}^{(i)} - \boldsymbol{y}^{(i)}\|^2/2$。然而，想要预测分类结果正确，我们其实并不需要预测概率完全等于标签概率。例如，在图像分类的例子里，如果$y^{(i)} = 3$，那么我们只需要$\hat{y}_3^{(i)}$比其他两个预测值$\hat{y}_1^{(i)}$和$\hat{y}_2^{(i)}$大就行了。即使$\hat{y}_3^{(i)}$值为0.6，不管其他两个预测值为多少，类别预测均正确。而平方损失则过于严格，例如$\hat{y}_1^{(i)} = \hat{y}_2^{(i)} = 0.2$比$\hat{y}_1^{(i)} = 0, \hat{y}_2^{(i)} = 0.4$的损失要小很多，虽然两者都有同样正确的分类预测结果。

改善上述问题的一个方法是使用更适合衡量两个概率分布差异的测量函数。其中，交叉熵（cross entropy）是一个常用的衡量方法：

$$H\left(\boldsymbol{y}^{(i)}, \hat{\boldsymbol{y}}^{(i)}\right) = -\sum_{j=1}^{q} y_j^{(i)} \log \hat{y}_j^{(i)},$$

其中带下标的$y_j^{(i)}$是向量$\boldsymbol{y}^{(i)}$中非0即1的元素，需要注意将它与样本i类别的离散数值，即不带下标的$y^{(i)}$区分。在上式中，我们知道向量$\boldsymbol{y}^{(i)}$中只有第$y^{(i)}$个元素$y_{y^{(i)}}^{(i)}$为1，其余全为0，于是$H(\boldsymbol{y}^{(i)}, \hat{\boldsymbol{y}}^{(i)}) = -\log \hat{y}_{y^{(i)}}^{(i)}$。也就是说，交叉熵只关心对正确类别的预测概率，因为只要其值足够大，就可以确保分类结果正确。当然，遇到一个样本有多个标签时，例如图像里含有不止一个物体时，我们并不能做这一步简化。但即便对于这种情况，交叉熵同样只关心对图像中出现的物体类别的预测概率。

假设训练数据集的样本数为n，交叉熵损失函数定义为

$$\ell(\boldsymbol{\Theta}) = \frac{1}{n} \sum_{i=1}^{n} H\left(\boldsymbol{y}^{(i)}, \hat{\boldsymbol{y}}^{(i)}\right),$$

其中$\boldsymbol{\Theta}$代表模型参数。同样地，如果每个样本只有一个标签，那么交叉熵损失可以简写成$\ell(\boldsymbol{\Theta}) = -(1/n) \sum_{i=1}^{n} \log \hat{y}_{y^{(i)}}^{(i)}$。从另一个角度来看，我们知道最小化$\ell(\boldsymbol{\Theta})$等价于最大化$\exp(-n\ell(\boldsymbol{\Theta})) = \prod_{i=1}^{n} \hat{y}_{y^{(i)}}^{(i)}$，即最小化交叉熵损失函数等价于最大化训练数据集所有标签类别的联合预测概率。

思考：

（1）交叉熵损失、信息熵、KL散度的关系是什么

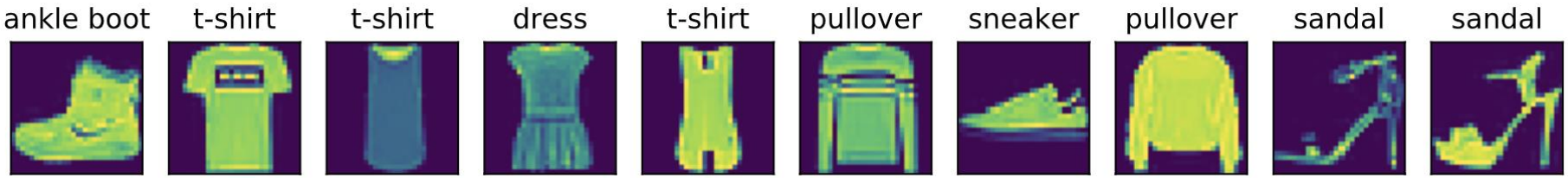
（2）对于分类问题，极大似然估计是否等价于交叉熵损失函数

3.5 图像分类数据集（Fashion-MNIST）

在介绍softmax回归的实现前我们先引入一个多类图像分类数据集。它将在后面的章节中被多次使用，以方便我们观察比较算法之间在模型精度和计算效率上的区别。图像分类数据集中最常用的是手写数字识别数据集MNIST[1]。但大部分模型在MNIST上的分类精度都超过了95%。为了更直观地观察算法之间的差异，我们将使用一个图像内容更加复杂的数据集Fashion-MNIST[2]（这个数据集也比较小，只有几十M，没有GPU的电脑也能吃得消）。

本节我们将使用torchvision包，它是服务于PyTorch深度学习框架的，主要用来构建计算机视觉模型。torchvision主要由以下几部分构成：

1. `torchvision.datasets`：一些加载数据的函数及常用的数据集接口；
2. `torchvision.models`：包含常用的模型结构（含预训练模型），例如AlexNet、VGG、ResNet等；
3. `torchvision.transforms`：常用的图片变换，例如裁剪、旋转等；
4. `torchvision.utils`：其他的一些有用的方法。



3.5.1 获取数据集

首先导入本节需要的包或模块。

Copy to clipboard

```
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import time
import sys
sys.path.append("..") # 为了导入上层目录的d2lzh_pytorch
import d2lzh_pytorch as d2l
```

下面，我们通过torchvision的 `torchvision.datasets` 来下载这个数据集。第一次调用时会自动从网上获取数据。我们通过参数 `train` 来指定获取训练数据集或测试数据集（testing data set）。测试数据集也叫测试集（testing set），只用来评价模型的表现，并不用来训练模型。

```
mnist_train =
torchvision.datasets.FashionMNIST(root='~/Datasets/FashionMNIST',
train=True, download=True, transform=transforms.ToTensor())
mnist_test =
torchvision.datasets.FashionMNIST(root='~/Datasets/FashionMNIST',
train=False, download=True, transform=transforms.ToTensor())
```

下面，我们通过torchvision的 `torchvision.datasets` 来下载这个数据集。第一次调用时会自动从网上获取数据。我们通过参数 `train` 来指定获取训练数据集或测试数据集（testing data set）。测试数据集也叫测试集（testing set），只用来评价模型的表现，并不用来训练模型。

另外我们还指定了参数 `transform = transforms.ToTensor()` 使所有数据转换为 `Tensor` ，如果不进行转换则返回的是PIL图片。 `transforms.ToTensor()` 将尺寸为 (H x W x C) 且数据位于[0, 255]的PIL图片或者数据类型为 `np.uint8` 的NumPy数组转换为尺寸为(C x H x W)且数据类型为 `torch.float32` 且位于[0.0, 1.0]的 `Tensor` 。

上面的 `mnist_train` 和 `mnist_test` 都是 **torch.utils.data.Dataset** 的子类，所以我们可以用 `len()` 来获取该数据集的大小，还可以用下标来获取具体的一个样本。训练集中和测试集中的每个类别的图像数分别为6,000和1,000。因为有10个类别，所以训练集和测试集的样本数分别为60,000和10,000。

Copy to clipboard

```python
print(type(mnist_train))
print(len(mnist_train), len(mnist_test))
```

输出：

```
<class 'torchvision.datasets.mnist.FashionMNIST'>
60000 10000
```

我们可以通过下标来访问任意一个样本:

```python
feature, label = mnist_train[0]
print(feature.shape, label)  # Channel x Height x Width
```

输出:

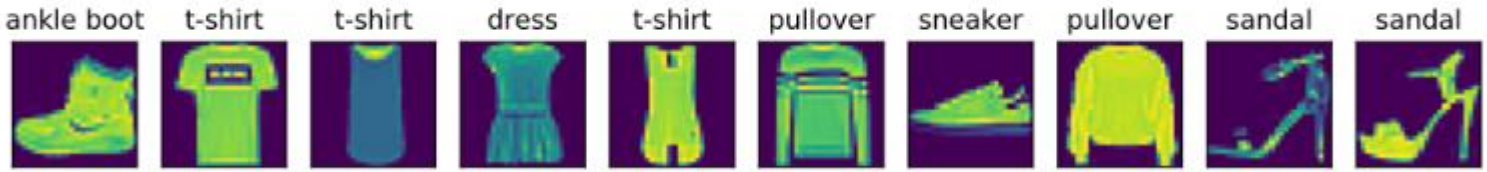
```
torch.Size([1, 28, 28]) tensor(9)
```

变量 `feature` 对应高和宽均为28像素的图像。由于我们使用了 `transforms.ToTensor()` ，所以每个像素的数值为[0.0, 1.0]的32位浮点数。需要注意的是， `feature` 的尺寸是 (C x H x W) 的，而不是 (H x W x C)。第一维是通道数，因为数据集中是灰度图像，所以通道数为1。后面两维分别是图像的高和宽。

Fashion-MNIST中一共包括了10个类别，分别为t-shirt（T恤）、trouser（裤子）、pullover（套衫）、dress（连衣裙）、coat（外套）、sandal（凉鞋）、shirt（衬衫）、sneaker（运动鞋）、bag（包）和ankle boot（短靴）。以下函数可以将数值标签转成相应的文本标签。

现在，我们看一下训练数据集中前10个样本的图像内容和文本标签。

```python
X, y = [], []
for i in range(10):
    X.append(mnist_train[i][0])
    y.append(mnist_train[i][1])
show_fashion_mnist(X, get_fashion_mnist_labels(y))
```



3.5.2 读取小批量

我们将在训练数据集上训练模型，并将训练好的模型在测试数据集上评价模型的表现。前面说过，`mnist_train` 是 `torch.utils.data.Dataset` 的子类，所以我们可以将其传入 `torch.utils.data.DataLoader` 来创建一个读取小批量数据样本的DataLoader实例。

在实践中，数据读取经常是训练的性能瓶颈，特别当模型较简单或者计算硬件性能较高时。PyTorch的 `DataLoader` 中一个很方便的功能是允许使用多进程来加速数据读取。这里我们通过参数 `num_workers` 来设置4个

```
batch_size = 256
 if sys.platform.startswith('win'): num_workers = 0 # 0表示不用额外的进程来加速读取数据
else: num_workers = 4
 train_iter = torch.utils.data.DataLoader(mnist_train, batch_size=batch_size, shuffle=True, num_workers=num_workers)
 test_iter = torch.utils.data.DataLoader(mnist_test, batch_size=batch_size, shuffle=False, num_workers=num_workers)
```

下面我们使用PyTorch来实现上一节中的多层感知机。首先导入所需的包或模块。

```
import torch
from torch import nn
from torch.nn import init
import numpy as np
import sys
sys.path.append("..")
import d2lzh_pytorch as d2l
```

3.10.1 定义模型

和softmax回归唯一的不同在于，我们多加了一个全连接层作为隐藏层。它的隐藏单元个数为256，并使用ReLU函数作为激活函数。

Copy to clipboard

```python
num_inputs, num_outputs, num_hiddens = 784, 10, 256

net = nn.Sequential(
        d2l.FlattenLayer(),
        nn.Linear(num_inputs, num_hiddens),
        nn.ReLU(),
        nn.Linear(num_hiddens, num_outputs),
        )

for params in net.parameters():
    init.normal_(params, mean=0, std=0.01)
```

3.10.2 读取数据并训练模型

我们使用与3.7节中训练softmax回归几乎相同的步骤来读取数据并训练模型。

> 注：由于这里使用的是PyTorch的SGD而不是d2lzh_pytorch里面的sgd，所以就不存在3.9节那样学习率看起来很大的问题了。

```
batch_size = 256
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size)
loss = torch.nn.CrossEntropyLoss()

optimizer = torch.optim.SGD(net.parameters(), lr=0.5)

num_epochs = 5
d2l.train_ch3(net, train_iter, test_iter, loss, num_epochs, batch_size, None, None, optimizer)
```

输出：

```
epoch 1, loss 0.0030, train acc 0.712, test acc 0.744
epoch 2, loss 0.0019, train acc 0.823, test acc 0.821
epoch 3, loss 0.0017, train acc 0.844, test acc 0.842
epoch 4, loss 0.0015, train acc 0.856, test acc 0.842
epoch 5, loss 0.0014, train acc 0.864, test acc 0.818
```

参考文献

[1]《统计学习方法》，清华大学出版社，李航著，2019年出版

[2]《机器学习》，清华大学出版社，周志华著，2016年出版

[3] Andrew Ng. Machine Learning[EB/OL]. Stanford University,2014. https://www.coursera.org/course/ml

[4] Christopher M. Bishop, Pattern Recognition and Machine Learning, Springer-Verlag, 2006

[5] Minsky, Marvin and Papert, Seymour. Perceptrons : An Introduction to Computational Geometry. The MIT Press, 1969.

[6] DE Rumelhart, Hinton G E, Williams R J. Learning Representations by Back Propagating Errors[J]. Nature, 1986, 323(6088):533-536.

[7] Bishop C M. Neural Networks for Pattern Recognition[J]. Advances in Computers, 1993, 37:119-166.

[8] Lecun Y, Bengio Y. Convolutional Networks for Images, Speech, and Time-Series[J]. Handbook of Brain Theory & Neural Networks, 1995.